

Alocação de Computação de Inferência na Síntese de Programas com Modelos de Linguagem

Anonymous submission

Resumo

Escalar a computação em tempo de inferência tornou-se o principal eixo de ganho em síntese de programas por modelos de linguagem, e um mesmo orçamento de chamadas admite duas alocações incompatíveis: *diversificar*, gerando N candidatos independentes, ou *iterar*, revisando um único candidato com feedback. A literatura relata ganhos para a segunda via, mas também falhas sistemáticas de autocorreção; as duas leituras são difíceis de separar porque, sem informação externa, o revisor não sabe que errou. Este trabalho remove esse confundimento por construção, concedendo à via iterativa uma vantagem que ela não teria em produção: um Agente Crítico que enxerga o gabarito do par de teste e devolve apenas contradições em linguagem natural, sem propor solução e sem participar da seleção do candidato final. O crítico opera assim como um limite superior otimista da via iterativa. Avaliamos as duas alocações no ARC-AGI-1 em um desenho pareado com orçamento igualado de 7 chamadas por tarefa. Em 270 tarefas, a revisão guiada por oráculo **não** superou a amostragem independente ($-0,7$ pp; McNemar exato $p = 0,8877$), e o intervalo de confiança limita qualquer vantagem do crítico a $+4,3$ pp. Uma decomposição fatorial subsequente, com dois críticos que variam isoladamente o acesso ao oráculo e a forma do feedback, também não separa as condições: **nenhuma** das 5 comparações pré-registradas sobrevive à correção de Bonferroni. Dois achados metodológicos independem do sinal do efeito: metade das vitórias de ambas as condições provém de uma chamada comum às duas, reduzindo o n efetivo de 270 para 206 tarefas, e a direção da diferença inverte entre $n = 30$ e $n = 60$, o que quantifica o tamanho de amostra abaixo do qual comparações desse tipo não são interpretáveis.

1 Introdução

Durante uma década, os ganhos em modelos de linguagem vieram de escalar parâmetros e dados de treino. O eixo mudou: hoje uma fração substancial do desempenho em tarefas de raciocínio vem de escalar a computação no momento da inferência — gastar mais chamadas, mais tokens e mais tempo por problema (Snell et al. 2024; Brown et al. 2024). Isso converte uma questão antes implícita em uma decisão

de engenharia com consequência direta de custo: dado um orçamento fixo de chamadas a um modelo, como alocá-lo?

Há duas respostas estruturalmente distintas. A primeira é **diversificar**: gerar N programas independentes e selecionar o melhor por um verificador barato. É a receita de *pass@k* em geração de código (Chen et al. 2021), da filtragem massiva do AlphaCode (Li et al. 2022) e da autoconsistência em raciocínio (Wang et al. 2023). A segunda é **iterar**: gastar o mesmo orçamento revisando um único candidato à luz de feedback, como em *self-debugging* (Chen et al. 2024), SELF-REFINE (Madaan et al. 2023) e REFLEXION (Shinn et al. 2023).

A evidência sobre a segunda via é contraditória. Os trabalhos originais relatam ganhos expressivos; réplicas controladas encontram que grande parte do ganho desaparece quando o orçamento é igualado, ou quando o modelo precisa detectar sozinho que errou (Huang et al. 2024; Stechly, Marquez, and Kambhampati 2023; Olausson et al. 2024). A explicação candidata mais simples é que autocrítica não é crítica: um modelo sem acesso a informação externa sobre o próprio erro tende a reafirmar a resposta que já produziu. Sob essa hipótese, o fator limitante não é a estratégia iterativa em si, mas a qualidade do canal de feedback.

Este trabalho testa essa hipótese maximizando exatamente a variável que ela aponta. Nosso Agente Crítico enxerga o gabarito do par de teste, que o Gerador nunca vê, e sua função é restrita por construção: não propõe solução, não escreve código e não reproduz grades, limitando-se a apontar em que pontos a regra enunciada e o programa candidato contradizem a realidade. Um filtro automático redige qualquer grade ou trecho de código que escape em sua saída, e o gabarito nunca participa do critério de parada nem da seleção do candidato final.

O desenho é, portanto, deliberadamente favorável à via iterativa: o crítico dispõe de informação privilegiada que nenhum sistema autônomo teria e ainda assim consome o mesmo orçamento de chamadas que a amostragem. Ele funciona como um **limite superior otimista**, o que torna a leitura do resultado informativa nos dois sentidos. Se o crítico vencer, o gargalo da autocorreção é informacional, e investir em verificadores externos se justifica. Se não vencer — mesmo com o gabarito disponível —, a vantagem atribuída ao feedback iterativo fica sob suspeita muito além deste arranjo específico.

Avaliamos no ARC-AGI-1 (Chollet 2019), um conjunto de tarefas de raciocínio abstrato em que cada item traz alguns pares entrada $\rightarrow$ saída de demonstração e exige inferir a transformação subjacente. O domínio é adequado à questão por três razões. A resposta é verificável exatamente — a grade de saída está inteiramente correta ou incorreta —, o que elimina a ambiguidade de métricas parciais. Os pares de demonstração fornecem um verificador barato e honesto que ambas as estratégias podem usar sem tocar no gabarito. E o *benchmark* permanece resistente a modelos de propósito geral (Chollet et al. 2024; Gendron et al. 2024), deixando espaço de medição que *benchmarks* saturados não deixam.

Contribuições.

- Um protocolo pareado com orçamento de chamadas igualado que isola a decisão diversificar vs. iterar de confundimentos de recursos, com salvaguardas explícitas e auditáveis contra contaminação pelo oráculo (Seções 3 e 3.4).
- Um resultado nulo com limite superior estreito em 270 tarefas: não apenas a ausência de efeito detectado, mas a delimitação de qualquer efeito a menos de 4,3 pp (Seção 4.1).
- Uma decomposição fatorial do mecanismo em dois eixos — acesso ao oráculo e forma do feedback — com uma família de 5 comparações pré-registradas sob correção de Bonferroni (Seção 4.3).
- Dois achados metodológicos transferíveis a estudos do gênero: o *desconto da primeira geração*, que mostra que metade das vitórias não pertence a nenhuma das estratégias comparadas (Seção 4.2), e a demonstração empírica de inversão de sinal entre $n = 30$ e $n = 60$ (Seção 4.4).

Em síntese, sob orçamento igualado a revisão guiada por oráculo **não** superou a amostragem independente, e **nenhuma** das 5 comparações pré-registradas sobrevive à correção de Bonferroni. O crítico cumpre o que promete no nível do mecanismo — atinge a mesma consistência com os pares de demonstração usando quase metade dos programas — sem que isso se converta em acerto no par de teste.

2 Trabalho Relacionado

Alocação de orçamento em largura. Amostrar muitos candidatos e filtrar é a estratégia de referência em síntese de programas com modelos de linguagem desde o Codex (Chen et al. 2021). Li et al. (2022) levaram a ideia ao limite, amostrando ordens de magnitude mais candidatos do que seriam submetidos e filtrando por execução em testes. Wang et al. (2023) demonstraram o análogo em raciocínio, agregando cadeias amostradas por votação. Trabalhos recentes caracterizam a curva: a cobertura cresce de forma previsível com o número de amostras (Brown et al. 2024), e há regimes em que alocar computação de inferência supera aumentar o modelo (Snell et al. 2024). O elo fraco dessa família é o *selector* — como escolher entre os N candidatos sem acesso à resposta —, endereçado na literatura por verificadores aprendidos sobre a resposta final (Cobbe et al. 2021) ou sobre passos intermediários (Lightman et al. 2024). Nossa condição

sampling é uma instância deliberadamente simples dessa família, com o verificador mais honesto que o ARC oferece sem custo adicional — os próprios pares de demonstração —, o que mantém a comparação livre de qualquer componente treinado.

Alocação de orçamento em profundidade. A via iterativa realimenta o modelo com o resultado da execução ou com uma crítica textual e solicita uma revisão. Chen et al. (2024) usam a saída do interpretador como feedback; Madaan et al. (2023) e Shinn et al. (2023) usam crítica gerada pelo próprio modelo. A contestação foi rápida: Huang et al. (2024) argumentam que modelos não corrigem raciocínio sem sinal externo, Stechly, Marquez, and Kambhampati (2023) mostram que a melhora aparente provém do verificador embutido no protocolo e não da autocritica, e Olausson et al. (2024) constatam que o auto-reparo em código só compensa quando o crítico é mais capaz que o gerador. Nosso desenho toma a variável que essa literatura identifica como determinante — a qualidade do sinal externo — e a maximiza.

Contraexemplos como canal de feedback. Guiar síntese por contraexemplos antecede os modelos de linguagem: em CEGIS (*Counterexample-Guided Inductive Synthesis*), um verificador devolve ao sintetizador uma entrada concreta em que o candidato falha, e o laço converge sobre uma especificação (Solar-Lezama et al. 2006; Alur et al. 2013; Gulwani, Polozov, and Singh 2017). Nossa condição `critic_cegis` transpõe esse protocolo para o registro de linguagem natural, restringindo a crítica a um contraexemplo e a uma classe de correção de vocabulário fechado, o que permite variar a forma do feedback mantendo o conteúdo informacional constante.

ARC-AGI e síntese de programas. O ARC foi proposto como medida de eficiência de aquisição de habilidade (Chollet 2019) e resiste a modelos gerais (Gendron et al. 2024; Mirchandani et al. 2023). Abordagens competitivas o tratam como síntese de programas guiada por busca sobre hipóteses em linguagem natural (Wang et al. 2024) ou por amostragem massiva de programas (Greenblatt 2024), na linhagem de sistemas de síntese indutiva como o DreamCoder (Ellis et al. 2021). Não propomos um novo estado da arte no *benchmark*: reportamos uma medição controlada de alocação de orçamento dentro de um arcabouço de síntese fixo.

3 Metodologia

3.1 Formulação do Problema

Uma tarefa t do ARC-AGI-1 é um par $(\mathcal{D}_t, (x_t^*, y_t^*))$, em que $\mathcal{D}_t = \{(x_i, y_i)\}_{i=1}^k$ são os pares de demonstração e (x_t^*, y_t^*) é o par de teste. Grades são matrizes de inteiros em $\{0, \dots, 9\}$ com dimensões variáveis.

Um *solucionador* é um procedimento que, recebendo $\mathcal{D}_t$ e x_t^* , produz um programa π em Python. A tarefa é considerada resolvida se e somente se $\pi(x_t^*) = y_t^*$ com igualdade exata da grade inteira. A métrica reportada é a acurácia $\text{acc} = \frac{1}{|T|} \sum_{t \in T} \mathbf{1}[\pi_t(x_t^*) = y_t^*]$.

O recurso controlado é o **orçamento de chamadas** B : o número máximo de requisições ao modelo de linguagem

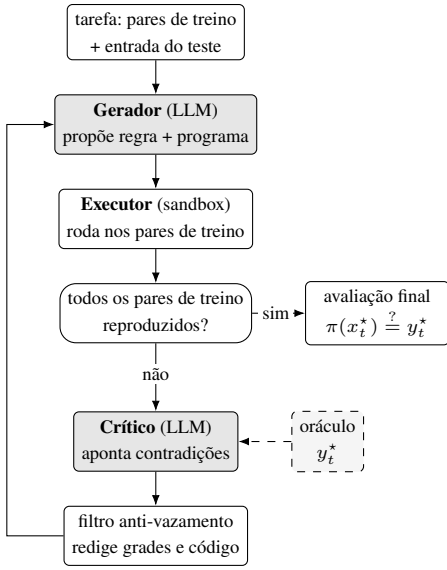


Figura 1: Arquitetura do solucionador. O Gerador nunca vê y_t^* em nenhuma condição; o Crítico o vê (linha tracejada) apenas nas condições que o concedem. A restrição central do desenho é que y_t^* não decide nada: o critério de parada e a seleção do candidato final consideram apenas os pares de demonstração, e a avaliação final é executada uma única vez, ao término do laço, exclusivamente para medir.

que o solucionador pode emitir por tarefa, somando todos os papéis de agente. Comparamos duas políticas $\mathcal{S}$ (diversificar) e $\mathcal{C}$ (iterar) de gasto de B , e a hipótese sob teste é

$$\Delta = \mathbb{E}_t[\mathbf{1}[\mathcal{C} \text{ resolve } t]] - \mathbb{E}_t[\mathbf{1}[\mathcal{S} \text{ resolve } t]] > 0, \quad (1)$$

com B mantido idêntico nas duas políticas. Essa igualdade é o controle experimental central da Equação 1: sem ela, a condição com mais recursos venceria por dispor de mais recursos, e a diferença observada não seria atribuível à estratégia.

3.2 Arquitetura do Solucionador

A Figura 1 descreve o laço comum a todas as condições. O Gerador recebe os pares de demonstração e a entrada de teste e propõe uma regra em linguagem natural acompanhada de um programa. O Executor roda o programa em *sandbox* sobre os pares de demonstração. Se todos forem reproduzidos, o laço termina; caso contrário, e havendo orçamento, o Crítico examina o candidato e devolve um feedback que passa pelo filtro anti-vazamento antes de retornar ao Gerador. As condições diferem apenas em como consomem o orçamento dentro desse laço e no que o Crítico enxerga.

3.3 Condições Experimentais

sampling (diversificar). Aloca o orçamento em largura. Cada uma das B chamadas é uma conversa nova com o mesmo *prompt* inicial: o modelo não sabe que já tentou antes, não vê o resultado da execução e não recebe crítica. Ao final, vence o candidato que reproduz mais pares de demonstração.

critic (iterar sob oráculo). Aloca o orçamento em profundidade. O Gerador propõe regra e programa; o Crítico, que enxerga y_t^* , aponta contradições em prosa livre; o Gerador revisa com o histórico completo da conversa. Cada ciclo custa duas chamadas, de modo que com $B = 7$ resultam 3,12 programas contra os 5,33 da amostragem. A hipótese é que a densidade informacional do feedback compense a redução no número de tentativas.

A adoção de *best-of-N* como linha de base é deliberada. Nada obriga quem dispõe de B chamadas a gastá-las revisando um único programa em série; medir a intervenção contra uma alternativa que aloca o orçamento pior produziria uma vantagem que informa mais sobre o comparativo escolhido do que sobre o método.

Decomposição do mecanismo. A comparação entre essas duas condições responde a uma questão pragmática, mas não a uma causal: elas diferem em quatro eixos simultâneos — existência de feedback, acesso ao gabarito, orçamento gasto em paralelo ou em série, e temperatura. Para separar as duas dimensões de maior interesse, definimos duas condições adicionais, cada uma variando um único eixo em relação a *critic* (Figura 2):

- *critic_no_oracle* — recebe os pares de demonstração, a regra, o código e o relatório de execução, mas nunca y_t^* . Isola quanto do efeito de uma crítica estruturada decorre apenas da estrutura, sem informação privilegiada.
- *critic_cegis* — recebe o mesmo que *critic*, inclusive y_t^* , mas responde com um único contraexemplo e uma classe de correção em vocabulário fechado (MISSING_CASE, WRONG_TRANSFORM, WRONG_GEOMETRY, WRONG_COLOR_MAP, WRONG_SCOPE, OTHER), em vez de prosa livre. Isola o efeito da forma do feedback com o acesso à informação mantido constante.

As duas são condições paralelas e independentes, não um *pipeline* sequencial. Um *pipeline* (Gerador → crítico sem oráculo → crítico de contraexemplo) foi considerado e descartado por variar as duas dimensões simultaneamente, o que impediria atribuir o efeito a um mecanismo específico. A Tabela 1 consolida a definição operacional das quatro condições.

3.4 Salvaguardas Contra Contaminação pelo Oráculo

Conceder o gabarito a um agente cria três riscos distintos, tratados por mecanismos distintos.

O gabarito não seleciona. O critério de parada é idêntico nas quatro condições: o candidato é aceito quando reproduz todos os pares de demonstração, e o candidato final é o melhor por esse mesmo critério. O par de teste é executado uma única vez, ao final, apenas para medir. Se o laço parasse quando o código acertasse o teste, a taxa de acerto passaria a ser um limite superior de oráculo, não uma medida de desempenho. Essa é a distinção entre usar o gabarito para formular feedback e usá-lo para escolher a resposta; apenas a primeira está sob teste.

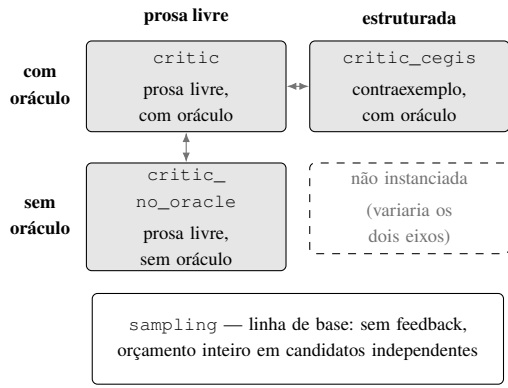


Figura 2: As quatro condições dispostas nos dois eixos que a decomposição pretende separar. As setas marcam as comparações que variam um único eixo: na horizontal, a forma do feedback com o acesso à informação constante; na vertical, o acesso à informação com a forma constante. A célula inferior direita não é instanciada porque variaria os dois eixos ao mesmo tempo.

	samp	critic	no_or	cegis
Feedback	—	prosa	prosa	estrut.
Vê y_t^*	não	sim	não	sim
Vê o código	—	sim	sim	sim
Histórico (Gerador)	não	sim	sim	sim
Memória (Crítico)	—	não	não	não
Temperatura	0,8	0,2	0,2	0,2
Chamadas por ciclo	1	2	2	2
Programas/tarefa	7	4	4	4
Desempate	1 ^a	últ.	últ.	últ.

Tabela 1: Definição operacional das quatro condições, abreviadas nos cabeçalhos (no_or = critic_no_oracle, cegis = critic_cegis). Todas competem sob o mesmo orçamento B de chamadas; variam a alocação desse orçamento e o que o Crítico enxerga. “Desempate” indica qual candidato vence quando há empate na consistência com os pares de demonstração.

O gabarito não vaza. O feedback do Crítico passa por um filtro automático antes de chegar ao Gerador: são redigidas linhas compostas apenas de inteiros (grades, listas de coordenadas) e qualquer código, seja em blocos cercados ou em linhas contendo `def`, `import`, `return`, `for` ou `while`. A resposta bruta é preservada nos registros para auditoria e cada redação é contabilizada. O vazamento deixa assim de depender da cooperação do modelo e torna-se um evento mensurável: uma contagem alta coloca a condição sob suspeita e é reportada.

O Crítico não resolve. O Crítico é reinstituído a cada rodada, sem histórico das anteriores. Sem memória, não pode construir uma solução ao longo do diálogo, e cada crítica é um julgamento independente do candidato apresentado. Um crítico capaz de acumular raciocínio seria um segundo solucionador, e a comparação deixaria de ser sobre alocação

Modelo	Acur. (%)	Ambas falham	Discord.
3.7-flash	~90	~0%	~0
3.5-flash-lite	26/28	66%	7 (14%)
3.1-flash-lite	13,5/16,2	81%	3 (8%)

Tabela 2: Seleção de modelo (prefixo `gemini-` omitido). Quando duas acurácias são reportadas, correspondem às duas condições comparadas. A coluna decisiva é a última: pares discordantes são as únicas observações que informam qual estratégia é melhor. `gemini-3.5-flash-lite` é o ponto mais informativo entre os modelos acessíveis, e foi escolhido por essa razão.

de orçamento.

3.5 Parâmetros de Configuração

Temperatura assimétrica. `sampling` opera a $T = 0,8$ e as condições de crítico a $T = 0,2$. Amostragem sem diversidade não é amostragem: a $T = 0,2$ as N amostras colapsam em programas quase idênticos e o *best-of-N* degenera em uma tentativa repetida N vezes. O custo metodológico é que a temperatura passa a ser mais um eixo de diferença entre as condições. Ela é constitutiva da estratégia e não uma variável de perturbação, mas parte de uma eventual vantagem de `sampling` decorre de explorar um espaço maior — o que é a estratégia operando, não um artefato.

Regra de desempate. Empates na consistência com os pares de demonstração são resolvidos pela última iteração nas condições de crítico e pela primeira amostra em `sampling`. Em `critic`, a iteração mais recente incorporou feedback, e preferi-la é coerente com a estratégia; em `sampling` não há progressão, e preferir a última faria com que amostrar mais deslocasse a escolha por si só, conferindo à estratégia uma vantagem independente da qualidade dos candidatos.

Paridade do orçamento. Uma condição de crítico só invoca o Crítico se ainda couberem duas chamadas, para não encerrar o orçamento em um feedback inutilizável. A última chamada de uma tarefa é, portanto, sempre do Gerador, e B deve ser ímpar: com $B = 6$, a condição usaria 5 chamadas enquanto `sampling` usaria 6.

Modelo único nos dois papéis. Gerador e Crítico usam o mesmo modelo. Se o Crítico fosse um modelo distinto, uma eventual vantagem poderia ser atribuída à capacidade do segundo modelo em vez da assimetria de informação, que é o que está sob teste.

3.6 Seleção do Modelo

Um desenho pareado só tem poder estatístico se existirem tarefas que uma estratégia resolve e a outra não. Modelos em qualquer dos extremos destroem essa propriedade de formas espelhadas: um modelo forte resolve a tarefa na primeira geração — idêntica nas duas condições — e produz apenas pares concordantes; um modelo fraco falha em ambas e também produz concordância. A Tabela 2 reporta a medição que orientou a escolha.

A seleção produziu ainda uma restrição de método que registramos por ser transferível: modelos com raciocínio interno não controlável são inadequados a este desenho. Um candidato testado passava, a partir da segunda iteração da condição de crítico, a consumir todo o teto de tokens de saída em raciocínio interno e a devolver texto vazio. O gatilho era especificamente a mensagem de revisão — exatamente o que distingue a condição sob teste. Um procedimento de aprovação que exercitasse apenas o *prompt* inicial teria aprovado o modelo; avaliar um modelo para este desenho exige exercitar as duas formas de *prompt*.

3.7 Protocolo de Análise Estatística

Desenho pareado. As mesmas tarefas rodam em todas as condições, sob o mesmo orçamento. A amostra é sorteada de *evaluation* com semente 20260814, declarada antes da execução, e a lista literal de identificadores é gravada no manifesto da rodada — é ela, e não o algoritmo de amostragem, que sustenta a reprodutibilidade.

Teste de McNemar exato. Em desenhos pareados, o que importa não é a diferença de acurácia agregada, mas quantas tarefas cada condição resolveu que a outra não (McNemar 1947). O teste descarta as tarefas em que as condições concordam e avalia se as discordâncias se dividem por acaso:

$$p = 2P(X \leq \min(a, b)), \quad X \sim \text{Binomial}(a + b, \frac{1}{2}), \quad (2)$$

com a e b as contagens de discordância a favor de cada condição. Adotamos a versão exata em vez da aproximação qui-quadrado, pouco confiável com poucas discordâncias.

Intervalo de confiança de Wilson. Calculado sobre a proporção de discordâncias a favor da intervenção (Wilson 1927):

$$\text{IC}_{95} = \frac{\hat{p} + \frac{z^2}{2n} \pm z \sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}, \quad (3)$$

preferido ao intervalo de Wald, que subestima o erro e pode produzir limites fora de $[0, 1]$. O intervalo é então convertido para pontos percentuais de acurácia. Reportá-lo é o que distingue a ausência de efeito detectado da ausência de capacidade de medição: um resultado nulo com limite superior estreito delimita quanto o efeito não pode ser.

Duas correções para dois problemas distintos. O estudo faz dois usos de múltiplos testes, cada um exigindo a correção do tipo apropriado.

Múltiplas análises ao longo do tempo. O Experimento 1 teve uma análise interina em 100 tarefas antes da análise principal em 270. Analisar os mesmos dados duas vezes oferece duas oportunidades de cruzar $\alpha = 0,05$: uma simulação sob H_0 com 20.000 réplicas mediu a taxa de erro tipo I subindo de 3,8% (apenas as 270 tarefas) para 6,3% (aceitando qualquer das duas análises). Por essa razão a análise principal foi fixada, antes da extensão da amostra, como sendo a de 270 tarefas; caso ambas fossem reivindicadas, aplicar-se-ia a correção de Pocock (Pocock 1977), com $\alpha = 0,0294$ em cada.

#	Par	Eixo isolado
1	samp vs. critic	hipótese principal
2	samp vs. no_or	crítica estruturada sem oráculo
3	samp vs. cegis	oráculo em forma de contraexemplo
4	critic vs. no_or	acesso ao gabarito, forma constante
5	critic vs. cegis	forma do feedback, acesso constante

Tabela 3: Família de 5 comparações pré-registrada antes da execução do Experimento 2. O par *no_or* vs. *cegis* fica deliberadamente fora por variar os dois eixos simultaneamente.

Múltiplas comparações simultâneas. O Experimento 2 pré-registrou uma família de 5 comparações sobre o mesmo conjunto de dados (Tabela 3), o que exige a correção de Bonferroni (Dunn 1961): $\alpha = 0,05/5 = 0,01$ por teste.

Exclusões declaradas. Uma tarefa interrompida por erro de API após as tentativas de reenvio é registrada e excluída da comparação pareada: um modelo indisponível não é evidência sobre o método, e contabilizá-la como falha concederia à outra condição um par discordante que ela não obteve.

3.8 Procedimento Experimental

Reportamos dois experimentos e um estudo piloto, todos sobre *evaluation* com a mesma semente 20260814 e $B = 7$, sob as condições da Seção 3.3 e o protocolo de análise da Seção 3.7.

Experimento 1 avalia a hipótese principal (Equação 1) em 270 tarefas, com as condições *sampling* e *critic* (21/08/2026, *commit* 5fdda8d). **Experimento 2** avalia a decomposição fatorial em 60 tarefas do mesmo sorteio, com as quatro condições (28/08/2026, *commit* d8e57e3); a condição *sampling* é reaproveitada do Experimento 1, por serem as mesmas tarefas sob a mesma semente, e não foi reexecutada. Um **estudo piloto** de 30 tarefas, com as quatro condições, precedeu o Experimento 2 e é reportado na Seção 4.4 como segunda medição da mesma configuração.

Duas versões do protocolo do Crítico. Os dois experimentos usam versões diferentes da mensagem enviada ao Crítico, e a distinção é material. Na versão **P1**, usada no Experimento 1, a mensagem continha a regra em linguagem natural e o relatório de execução nos pares de demonstração, mas *não* o código candidato: o Crítico julgava uma descrição do algoritmo, não o algoritmo, e não podia apontar divergências entre o código e a regra enunciada nem erros visíveis apenas no código. Na versão **P2**, usada no Experimento 2 e no estudo piloto, o código candidato é incluído. A correção não introduz vazamento: o filtro anti-vazamento atua sobre a saída do Crítico, nunca sobre a entrada, e o código candidato é a própria saída do Gerador. A mudança foi validada por inspeção das críticas brutas, que passaram a referenciar o comportamento do código explicitamente — algo estruturalmente impossível sob P1. Uma crítica representativa sob

Condição	Res.	Acur.	Treino	Prog.	Tokens
sampling	88/270	32,6%	35,6%	5,43	7,6 M
critic	86/270	31,9%	35,6%	3,30	12,6 M

Tabela 4: Experimento 1, resultados agregados. “Treino” é a fração de tarefas em que o candidato final reproduz todos os pares de demonstração — o alvo que ambas as estratégias de fato otimizam; “Prog.” é o número médio de programas gerados por tarefa.

	critic resolveu	critic falhou
sampling resolveu	62	26
sampling falhou	24	158

Tabela 5: Tabela de contingência pareada do Experimento 1. Apenas as células em negrito informam qual estratégia é melhor; as 50 discordâncias (18,5% das tarefas) dividem-se 24×26 .

P2:

“The stated rule and code incorrectly assume an intersection-based logical operation... the code implements a simpler neighborhood check for any cell adjacent to both a 4 and a 5, converting many incorrect cells...”

A escala reduzida do Experimento 2 relativamente ao Experimento 1 decorre de restrição de cota de API e é tratada explicitamente na Seção 4.4 e entre as limitações (Seção 5.3).

4 Resultados

4.1 Comparação Principal em Larga Escala

O Experimento 1 executou 1.812 chamadas em 20 minutos, com 7 chaves de API em paralelo. A Tabela 4 reporta os resultados agregados e a Tabela 5, a estrutura pareada.

Sob orçamento igualado, a revisão guiada por oráculo **não** superou a amostragem independente: a diferença é de $-0,7$ pp, com McNemar exato $p = 0,8877$ (Equação 2 com $a = 24$ e $b = 26$). O intervalo de confiança de Wilson (Equação 3) sobre a proporção de discordâncias a favor do Crítico, convertido para acurácia, é $-5,6$ a $+4,3$ pp.

As duas condições resolvem conjuntos quase coincidentes: 62 tarefas em comum, 158 em que ambas falham, e as 50 discordâncias divididas praticamente ao meio. A diferença chegou a inverter de sinal entre a análise interina de 100 tarefas e o conjunto de 270, comportamento compatível com ruído e não com efeito. O intervalo de confiança delimita qualquer vantagem do Crítico a 4,3 pp.

O mecanismo, contudo, opera no nível em que foi projetado. O Crítico atinge a mesma consistência com os pares de demonstração (35,6%) usando quase metade dos programas (3,30 contra 5,43 por tarefa): a revisão dirigida de fato converge mais rápido para programas consistentes. O que não se observa é a tradução disso em acerto no par de teste. A explicação candidata mais natural — que o Crítico induziria maior sobreajuste aos pares de demonstração — foi registrada como exploratória e testada nas tarefas novas iso-

	Vitórias	Na 1ª geração	Atribuíveis ao ciclo
sampling	88	44	44
critic	86	42	44

Tabela 6: Decomposição das vitórias do Experimento 1. Descontadas as obtidas antes de qualquer amostragem adicional ou crítica, o placar é 44×44 .

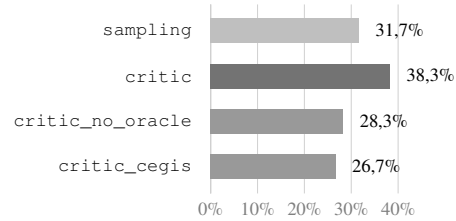


Figura 3: Acurácia por condição no Experimento 2 ($n = 60$). A ordenação observada favorece *critic*, mas nenhuma diferença é significativa (Figura 4) e a mesma ordenação se inverte em uma amostra de 30 tarefas (Seção 4.4).

ladamente, sem se replicar (23,3% na análise interina contra 15,2% na réplica, $p = 0,6173$).

4.2 Origem das Vitórias: o Desconto da Primeira Geração

A primeira chamada é idêntica nas duas condições — mesmo *prompt*, histórico vazio —, diferindo apenas na temperatura. Toda tarefa resolvida já nessa chamada é uma vitória anterior a qualquer ciclo de amostragem ou de crítica, e portanto não atribuível a nenhuma das estratégias.

Descontada a primeira geração (Tabela 6), o placar atribuível ao ciclo é 44×44 . Nas 206 tarefas que a primeira geração não resolveu — as únicas em que o experimento efetivamente discrimina entre as estratégias — o placar é 18×15 ($p = 0,7283$).

Este não é um artefato a eliminar: com temperaturas distintas por definição, a primeira amostra integra cada estratégia. Ele tem, porém, duas consequências mensuráveis. O poder estatístico efetivo é menor do que n sugere — aqui, 206 e não 270 tarefas. E um estudo que reportasse apenas a acurácia agregada atribuiria à estratégia metade de um desempenho originado em uma chamada comum a todas as condições comparadas.

4.3 Decomposição Fatorial do Mecanismo

As acurácias agregadas constam da Tabela 7 e da Figura 3; as comparações pareadas, da Tabela 8 e da Figura 4: **nenhuma** das 5 comparações pré-registradas sobrevive à correção de Bonferroni. A comparação mais próxima é *critic* vs. *critic_cegis*, com $p = 0,0391$, que cruzaria $\alpha = 0,05$ isoladamente mas não o $\alpha = 0,01$ da família pré-registrada. Sob o protocolo P2, a direção observada nesta amostra favorece *critic* sobre *sampling* em $+6,7$ pp, invertendo o sinal observado no estudo piloto.

A restrição de formato de *critic_cegis* foi obedecida de maneira consistente: todas as críticas emitidas (60/60 no

Condição	Res.	Acur.	Treino	Prog.	Tok.
sampling	19/60	31,7%	36,7%	5,33	25,8 k
critic	23/60	38,3%	41,7%	3,12	44,3 k
no_oracle	17/60	28,3%	33,3%	3,30	41,6 k
cegis	16/60	26,7%	30,0%	3,32	46,3 k

Tabela 7: Experimento 2, resultados agregados nas quatro condições. Colunas como na Tabela 4; “Tok.” é o consumo médio de tokens por tarefa.

Par	Discord.	p	IC 95%
samp-critic	6 (1×5)	0,2188	-1,3 a +9,4
samp-no_or	10 (6×4)	0,7539	-11,1 a +6,2
samp-cegis	7 (5×2)	0,4531	-9,7 a +3,3
critic-no_or	10 (8×2)	0,1094	-14,8 a +0,3
critic-cegis	9 (8×1)	0,0391	-14,4 a -1,9

Tabela 8: As 5 comparações pré-registradas do Experimento 2, com p de McNemar exato e intervalos em pontos percentuais. Sob Bonferroni ($\alpha = 0,01$), nenhuma é significativa. A comparação 5 cruzaria $\alpha = 0,05$ isoladamente, mas não a correção da família.

Experimento 2, 72/72 no piloto) trouxeram um rótulo válido de classe de correção, e o risco de o modelo escapar do vocabulário fechado não se concretizou. A adesão ao formato, contudo, não se traduziu em vantagem de acurácia, e a comparação mais próxima de significância aponta contra essa condição.

4.4 Estabilidade das Estimativas com o Tamanho da Amostra

No estudo piloto de 30 tarefas, *critic* apresentou acurácia abaixo de *sampling* (23,3% contra 36,7%); no Experimento 2, com o dobro de tarefas, apresentou acurácia acima (38,3% contra 31,7%). Em nenhum dos dois tamanhos há significância (Tabela 9, Figura 5).

A inversão ocorre com o mesmo código, o mesmo modelo, a mesma semente e o mesmo protocolo, o que a torna uma medida direta do piso de tamanho de amostra abaixo do qual comparações desse tipo não são interpretáveis. Trata-se do mecanismo que produz literaturas contraditórias: um estudo que parasse no piloto reportaria que o crítico prejudica; outro que parasse no Experimento 2 reportaria que o crítico ajuda; ambos com p não significativo, ambos reportáveis como observação direcional, e ambos ruído (Button et al. 2013; Gelman and Loken 2014; Ioannidis 2005).

4.5 Análise Qualitativa

Uma tarefa ilustra os dois eixos da decomposição simultaneamente. Ela requer conectar, por segmentos de reta, pares de pixels de mesma cor que compartilham linha ou coluna; o detalhe que invalida soluções ingênuas é que, no cruzamento de linhas de cores diferentes, a vertical tem precedência sobre a horizontal.

sampling falha nas 7 tentativas independentes, cada uma reinventando uma variação da mesma regra in-

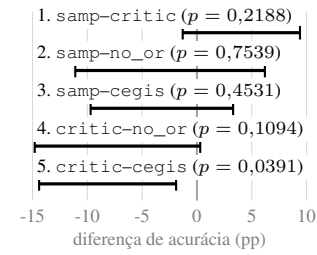


Figura 4: Intervalos de confiança de 95% das 5 comparações pré-registradas. A linha vertical marca a ausência de efeito. Todos os intervalos, exceto o da comparação 5, a cruzam; e o da comparação 5 não sobrevive à correção de Bonferroni ($\alpha = 0,01$). O que a figura evidencia é menos uma ordenação entre condições do que a largura dos intervalos: nesta escala de amostra, a medição não distingue as hipóteses.

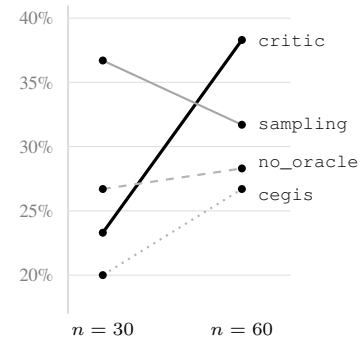


Figura 5: A mesma configuração medida em duas amostras. Em $n = 30$ tarefas, *critic* fica abaixo de *sampling*; em $n = 60$, fica acima. Nenhuma das duas diferenças é significativa. As leituras não se contradizem: são o mesmo sintoma de que, nesta escala, a estimativa de acurácia tem variância grande demais para revelar direção de forma confiável.

completa: sem informação sobre o erro, nenhuma delas chega a mencionar o comportamento nos cruzamentos. *critic_no_oracle* resolve em duas rodadas sem jamais ver o gabarito, com a seguinte crítica:

“The code faithfully implements the stated rule by independently drawing horizontal and vertical line segments for all matching non-zero colors, overwriting grid cells where different colors intersect without any prioritization.”

Trata-se de uma comparação entre código e regra, não com o gabarito, e é suficiente para localizar o problema. *critic* chega à mesma correção, também em duas rodadas: nesta tarefa, o acesso ao gabarito não trouxe vantagem sobre a ausência dele. *critic_cegis*, que dispõe do gabarito, falha; sua crítica mais informativa chega a mencionar “*preserving the proper color precedence*”, mas o vocabulário fechado não permite expressar qual orientação tem precedência sobre qual, e o Gerador alterna hipóteses sem convergir.

O caso constitui indício qualitativo alinhado ao quantitativo: ao menos aqui, a forma do feedback importou mais que o acesso à informação. Um segundo caso serve de contrapeso: em outra tarefa, *critic* e *critic_no_oracle*

Condição	Piloto ($n = 30$)	Exp. 2 ($n = 60$)
sampling	11/30 (36,7%)	19/60 (31,7%)
critic	7/30 (23,3%)	23/60 (38,3%)
no_oracle	8/30 (26,7%)	17/60 (28,3%)
cegis	6/30 (20,0%)	16/60 (26,7%)

Tabela 9: Tarefas resolvidas e acurácia nas duas medições da mesma configuração sob o protocolo P2. Em $n = 30$ as três condições de crítico ficam abaixo da linha de base; em $n = 60$, uma delas fica acima. Nenhuma diferença, em nenhuma das escalas, é significativa.

vencem na primeira geração, sem que nenhuma crítica seja invocada. Não é evidência do mecanismo, e sim o desconto da Seção 4.2 reaparecendo no nível do caso individual.

5 Discussão

5.1 Interpretação

A leitura defensável dos resultados é pragmática: sob orçamento de chamadas igualado e neste arranjo, alocar o orçamento em diversidade não é pior do que alocá-lo em revisão dirigida — nem mesmo quando a revisão é guiada por um oráculo indisponível a qualquer sistema autônomo. Como o desenho favorece a via iterativa por construção, o resultado é mais forte do que um empate entre esquemas equivalentes: indica que a informação privilegiada, isoladamente, não é o fator limitante.

Os resultados não sustentam, porém, a leitura causal de que a assimetria de informação seja irrelevante. As condições diferem em quatro eixos simultâneos, e a decomposição fatorial construída para separá-los opera em escala insuficiente para concluir. O que o Experimento 2 entrega é uma decomposição montada e pré-registrada, pronta para ser levada à escala do Experimento 1.

5.2 Implicações Práticas

A unidade do orçamento altera a conclusão. Contabilizamos o orçamento em chamadas. Contado em tokens, o quadro se deteriora para o Crítico: no Experimento 1 ele consome 66% mais tokens por tarefa resolvida (146 k contra 86 k) e no Experimento 2, 72% mais tokens por tarefa (44,3 k contra 25,8 k), porque o histórico acumulado e o gabarito completo trafegam a cada crítica. Sob orçamento medido em tokens — a unidade de custo efetiva — a comparação seria substancialmente menos favorável à via iterativa. Reportamos em chamadas por ser a unidade que torna o número de tentativas comparável, mas a decisão prática deve considerar ambas.

O regime em que a amostragem não converte orçamento em vantagem. Em uma fração substancial das tarefas, todos os candidatos amostrados empatam em zero pares de demonstração corretos; com o desempate pela primeira amostra, o *best-of-N* torna-se operacionalmente idêntico a uma tentativa única. Nas tarefas difíceis do ARC, o sinal de treino é fraco demais para ordenar candidatos. Esse é precisamente o regime em que um crítico com informação externa

deveria oferecer maior ganho, e onde uma intervenção futura tem mais a explorar do que no regime médio capturado pela acurácia agregada.

5.3 Limitações e Ameaças à Validade

1. **Confundimento múltiplo.** `sampling` e `critic` diferem em quatro eixos simultaneamente. Um resultado positivo sustentaria a leitura pragmática, não a causal. As condições do Experimento 2 endereçam dois desses eixos, em escala reduzida.
2. **O oráculo não é autônomo.** A condição `critic` usa informação indisponível em cenário real. O valor reportado mede a validação por oposição como limite superior, não um sistema utilizável em produção.
3. **Poder efetivo reduzido.** O desconto da primeira geração (Seção 4.2) reduz o n efetivo de 270 para 206 tarefas.
4. **Execução única por tarefa.** Sem repetições, parte da diferença observada é ruído de amostragem do modelo, ampliado em `sampling` pela temperatura alta. O pareamento mitiga, mas não elimina.
5. **Vazamento residual.** O filtro captura grades e código, não paráfrases (“a saída tem metade da altura”). A contagem de redações mede o que foi barrado, não o que passou; a inspeção manual dos *feedbacks* brutos nos casos discordantes complementa, sem garantir.
6. **Escala do Experimento 2.** 60 tarefas contra 270 — poder baixo por desenho. Nenhuma das 5 comparações deve ser lida como conclusiva, e a Seção 4.4 quantifica a razão.
7. **Diferença de protocolo entre os experimentos.** O Experimento 1 usou P1 e o Experimento 2 usou P2 (Seção 3.8). As duas comparações `sampling-critic` não são, portanto, réplicas exatas uma da outra.
8. **Dependência de modelo.** A acurácia absoluta depende fortemente do modelo (Tabela 2), e a Seção 3.6 mostra que a própria existência de pares discordantes também depende. As conclusões referem-se ao regime de capacidade de `gemin-3.5-flash-lite`, não a modelos em geral.

6 Conclusão

Sob orçamento fixo de chamadas a um modelo de linguagem para síntese de programas no ARC-AGI-1, a revisão guiada por oráculo **não** superou a amostragem independente, com intervalo de confiança que delimita qualquer vantagem a 4,3 pp em 270 tarefas. A decomposição do mecanismo em dois eixos — acesso ao oráculo e forma do feedback — em 60 tarefas não separou as condições: **nenhuma** das 5 comparações pré-registradas sobrevive à correção de Bonferroni. O crítico opera como mecanismo, atingindo a mesma consistência com os pares de demonstração com quase metade dos programas, sem que isso se converta em acerto no par de teste.

Três contribuições independem de qual condição venha a prevalecer em escala maior. A primeira é o protocolo: orçamento igualado, desenho pareado, oráculo que informa mas não seleciona, filtro de vazamento auditável e família

de comparações pré-registrada. A segunda é o desconto da primeira geração, que mostra que metade das vitórias em estudos desse formato não pertence a nenhuma das estratégias comparadas e reduz silenciosamente o poder efetivo. A terceira é a inversão de sinal documentada entre $n = 30$ e $n = 60$, que quantifica o piso de amostra abaixo do qual comparações entre estratégias de alocação de orçamento não são interpretáveis — e que explica, sem recorrer a má conduta, como a literatura sobre autocorreção iterativa acumulou resultados divergentes.

O passo natural é levar as quatro condições à escala do Experimento 1, com a decomposição já montada e pré-registrada. Duas direções adicionais nos parecem mais promissoras do que aumentar n isoladamente: medir o orçamento em tokens, que é a unidade de custo efetiva e altera o sinal esperado da comparação; e concentrar a medição no regime em que a amostragem comprovadamente não converte orçamento em vantagem — as tarefas em que todos os candidatos empatam em zero pares de demonstração corretos —, onde um crítico informado tem, em princípio, mais a oferecer.

Reprodutibilidade

O ambiente é gerenciado por `uv`, com interpretador fixado e versões de dependências travadas em arquivo de `lock` versionado. Cada rodada grava um manifesto com a lista literal de identificadores de tarefa, a semente, o orçamento, os modelos e o número (nunca a identidade) de chaves de API utilizadas, além de um registro `jsonl` por condição contendo todos os *prompts*, respostas brutas, contagens de token, códigos gerados e resultados de execução por tarefa. As respostas brutas do Crítico são preservadas antes da aplicação do filtro anti-vazamento, o que permite auditar independentemente tanto o que foi redigido quanto o que passou. Um modo de execução seca percorre o *pipeline* inteiro sem consumir API, para verificação de instalação. Os testes automatizados travam as propriedades críticas do desenho, incluindo a separação de informação entre os *prompts* do Gerador e do Crítico e a paridade de orçamento entre as condições.

Referências

Alur, R.; Bodík, R.; Juniwal, G.; Martin, M. M. K.; Raghothaman, M.; Seshia, S. A.; Singh, R.; Solar-Lezama, A.; Torlak, E.; and Udupa, A. 2013. Syntax-Guided Synthesis. In *Formal Methods in Computer-Aided Design (FMCAD)*, 1–8.

Brown, B.; Juravsky, J.; Ehrlich, R.; Clark, R.; Le, Q. V.; Ré, C.; and Mirhoseini, A. 2024. Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. arXiv:2407.21787.

Button, K. S.; Ioannidis, J. P. A.; Mokrysz, C.; Nosek, B. A.; Flint, J.; Robinson, E. S. J.; and Munafò, M. R. 2013. Power Failure: Why Small Sample Size Undermines the Reliability of Neuroscience. *Nature Reviews Neuroscience*, 14(5): 365–376.

Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; Pinto, H. P. d. O.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman,

G.; et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374.

Chen, X.; Lin, M.; Schärli, N.; and Zhou, D. 2024. Teaching Large Language Models to Self-Debug. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Chollet, F. 2019. On the Measure of Intelligence. arXiv:1911.01547.

Chollet, F.; Knoop, M.; Kamradt, G.; and Landers, B. 2024. ARC Prize 2024: Technical Report. arXiv:2412.04604.

Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; Hesse, C.; and Schulman, J. 2021. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168.

Dunn, O. J. 1961. Multiple Comparisons Among Means. *Journal of the American Statistical Association*, 56(293): 52–64.

Ellis, K.; Wong, C.; Nye, M.; Sablé-Meyer, M.; Morales, L.; Hewitt, L.; Cary, L.; Solar-Lezama, A.; and Tenenbaum, J. B. 2021. DreamCoder: Bootstrapping Inductive Program Synthesis with Wake-Sleep Library Learning. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*, 835–850.

Gelman, A.; and Loken, E. 2014. The Statistical Crisis in Science. *American Scientist*, 102(6): 460–465.

Gendron, G.; Bao, Q.; Witbrock, M.; and Dobbie, G. 2024. Large Language Models Are Not Strong Abstract Reasoners. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI-24)*, 6270–6278.

Greenblatt, R. 2024. Getting 50% (SoTA) on ARC-AGI with GPT-4o. <https://redwoodresearch.substack.com/p/getting-50-sota-on-arc-agi-with-gpt>. Accessed: 2026-08-28.

Gulwani, S.; Polozov, O.; and Singh, R. 2017. Program Synthesis. *Foundations and Trends in Programming Languages*, 4(1-2): 1–119.

Huang, J.; Chen, X.; Mishra, S.; Zheng, H. S.; Yu, A. W.; Song, X.; and Zhou, D. 2024. Large Language Models Cannot Self-Correct Reasoning Yet. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Ioannidis, J. P. A. 2005. Why Most Published Research Findings Are False. *PLoS Medicine*, 2(8): e124.

Li, Y.; Choi, D.; Chung, J.; Kushman, N.; Schrittwieser, J.; Leblond, R.; Eccles, T.; Keeling, J.; Gimeno, F.; Dal Lago, A.; et al. 2022. Competition-Level Code Generation with AlphaCode. *Science*, 378(6624): 1092–1097.

Lightman, H.; Kosaraju, V.; Burda, Y.; Edwards, H.; Baker, B.; Lee, T.; Leike, J.; Schulman, J.; Sutskever, I.; and Cobbe, K. 2024. Let’s Verify Step by Step. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegrefe, S.; Alon, U.; Dziri, N.; Prabhunoye, S.; Yang, Y.; et al. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 46534–46594.

- McNemar, Q. 1947. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika*, 12(2): 153–157.
- Mirchandani, S.; Xia, F.; Florence, P.; Ichter, B.; Driess, D.; Arenas, M. G.; Rao, K.; Sadigh, D.; and Zeng, A. 2023. Large Language Models as General Pattern Machines. In *Proceedings of the 7th Conference on Robot Learning (CoRL)*, 2498–2518.
- Olausson, T. X.; Inala, J. P.; Tan, C.; Gao, J.; and Solar-Lezama, A. 2024. Is Self-Repair a Silver Bullet for Code Generation? In *The Twelfth International Conference on Learning Representations (ICLR)*.
- Pocock, S. J. 1977. Group Sequential Methods in the Design and Analysis of Clinical Trials. *Biometrika*, 64(2): 191–199.
- Shinn, N.; Cassano, F.; Berman, E.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 8634–8652.
- Snell, C.; Lee, J.; Xu, K.; and Kumar, A. 2024. Scaling LLM Test-Time Compute Optimally Can Be More Effective Than Scaling Model Parameters. arXiv:2408.03314.
- Solar-Lezama, A.; Tancau, L.; Bodik, R.; Seshia, S.; and Saraswat, V. 2006. Combinatorial Sketching for Finite Programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 404–415.
- Stechly, K.; Marquez, M.; and Kambhampati, S. 2023. GPT-4 Doesn’t Know It’s Wrong: An Analysis of Iterative Prompting for Reasoning Problems. arXiv:2310.12397.
- Wang, R.; Zelikman, E.; Poesia, G.; Pu, Y.; Haber, N.; and Goodman, N. D. 2024. Hypothesis Search: Inductive Reasoning with Language Models. In *The Twelfth International Conference on Learning Representations (ICLR)*.
- Wang, X.; Wei, J.; Schuurmans, D.; Le, Q. V.; Chi, E. H.; Narang, S.; Chowdhery, A.; and Zhou, D. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *The Eleventh International Conference on Learning Representations (ICLR)*.
- Wilson, E. B. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*, 22(158): 209–212.